

Создание игры

На дисциплине «Программирование»

ЦАРЕГОРОДЦЕВ Георгий Павлович

Что такое GameDev?

GameDev – процесс создания игры: от разработки и дизайна до выпуска на рынок. Это могут быть игры для телефонов, консолей, компьютеров или других гаджетов.

GameDev – процесс создания игры: от разработки и дизайна до выпуска на рынок. Это могут быть игры для телефонов, консолей, компьютеров или других гаджетов.

Что есть в GameDev?

GameDev – процесс создания игры: от разработки и дизайна до выпуска на рынок. Это могут быть игры для телефонов, консолей, компьютеров или других гаджетов.

В GameDev также есть место:

- 1. Жанры игр**
- 2. Об игровых движках**
- 3. Архитектура**
4. Дизайн игрового окружения
5. Баланс
6. Интерфейсы. UX/UI
7. Тестирование и публикация
8. Монетизация

Жанры игр

1. **Сюжетные.** Игрок выступает в роли героя, действуя от первого лица.
2. **Экшены.** Действие развивается динамично, успех игрока зависит от его скорости реакции и способности быстро принимать тактические решения.
3. **Аркады.** Главная особенность — простое управление.
4. **Хорроры.** Игры-страшилки, которые держат в напряжении на протяжении всего игрового процесса.
5. **Платформеры.** Основа геймплея — лазать по лестницам, прыгать по платформам, собирать добычу.
6. **Файтинги.** Игры, посвящённые боевым искусствам: как вымышленным, так и реальным.
7. **Интерактивное кино.** Сюжет состоит из отснятых сцен с участием актёров или анимации. Игрок смотрит на мир от третьего лица и управляет действиями одного или нескольких главных героев.
8. **Казуальные.** Логические или интеллектуальные игры с простыми и понятными правилами.
9. **Квесты или приключенческие игры.** Упор сделан на умственные усилия и творческое мышление игроков. Среди классических заданий — поиск предметов на картинках, способов обойти препятствия, решение математических и логических задач.
10. **RPG (ролевые).** Игроку предоставляется полная свобода действий. Есть элементы планирования, чтобы сюжет в конечном счёте сложился как пазл.

Об игровых движках

Игровой движок — базовое программное обеспечение, обеспечивающее работоспособность геймплейных, графических и других элементов видеоигры. На базе одного и того же игрового движка можно создавать различные игры, «навешивая» на него соответствующие текстуры, модели персонажей и объектов окружения, внутриигровые правила и т.д.

Выделить можно следующие крупные движки:

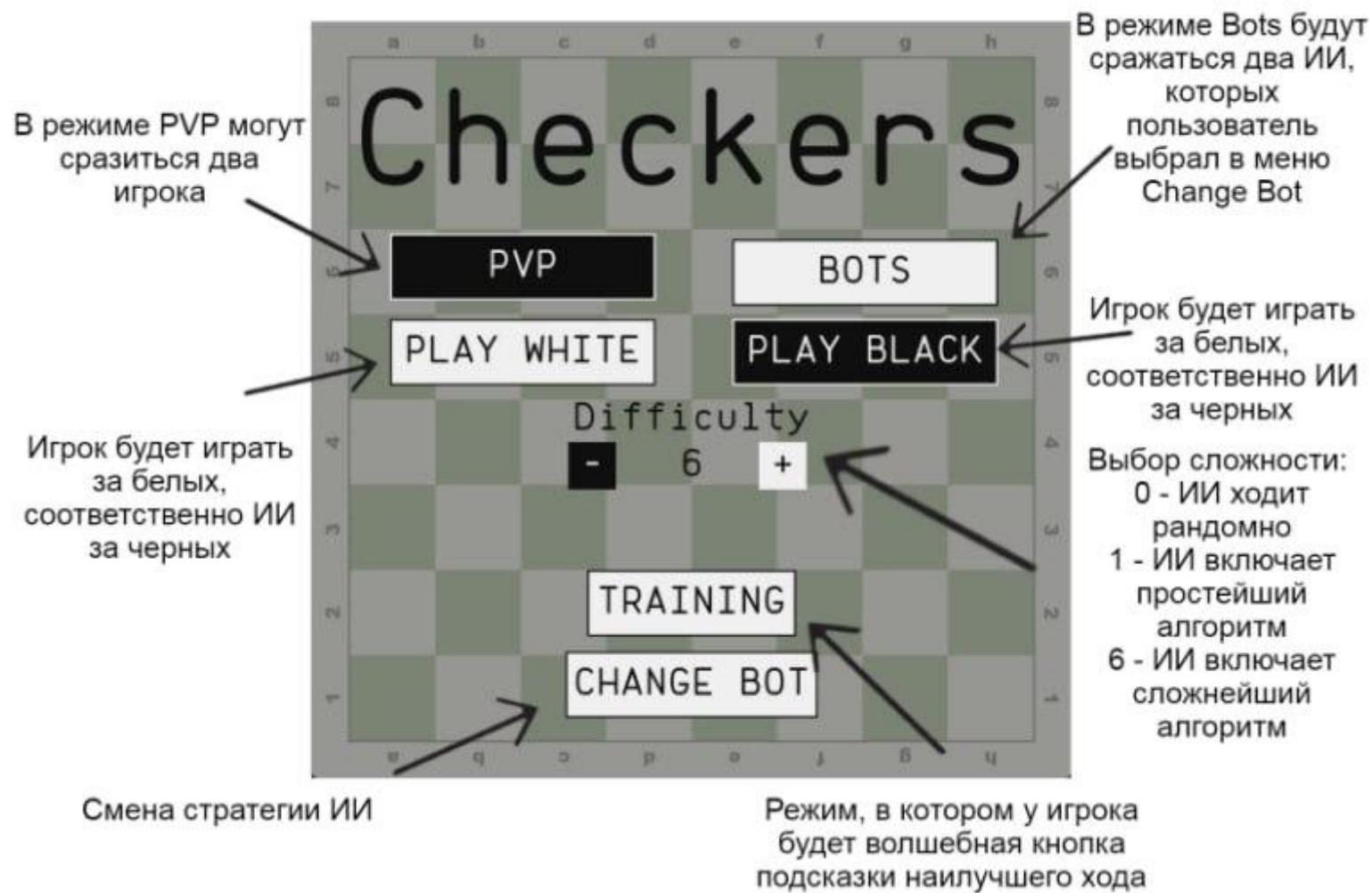
1. **Unity**
2. **Unreal Engine**
3. **CryEngine**
4. **Godot**
5. **Defold**
6. **Amazon Lumberyard**
7. **PyGame**

Игровые движки

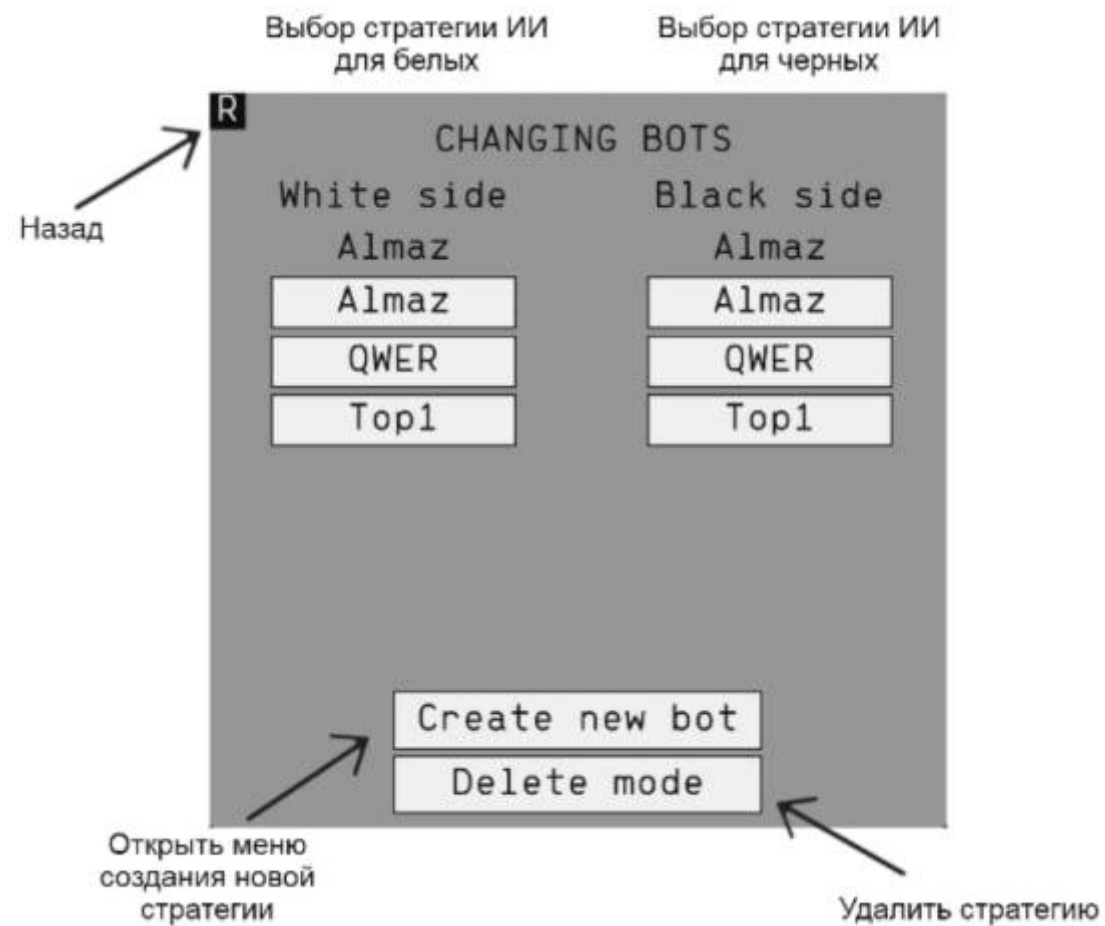
PyGame

PyGame – это библиотека для создания игр на языке программирования Python. В библиотеке собран набор программных средств, помогающих разрабатывать двумерные игры.

PyGame. Пример 1



PyGame. Пример 1



PyGame. Пример 1

Назад → **R**

Enter 20 weights for the checkerboard characteristics.
These weights are used to determine the quality of the position.
This influences the selection of the best move.

Поле ввода названия стратегии → Name

Поля ввода числовых значений для определения значимости того или иного параметра. На самом деле их намного больше, просто можно еще пролистать вниз колесиком мыши

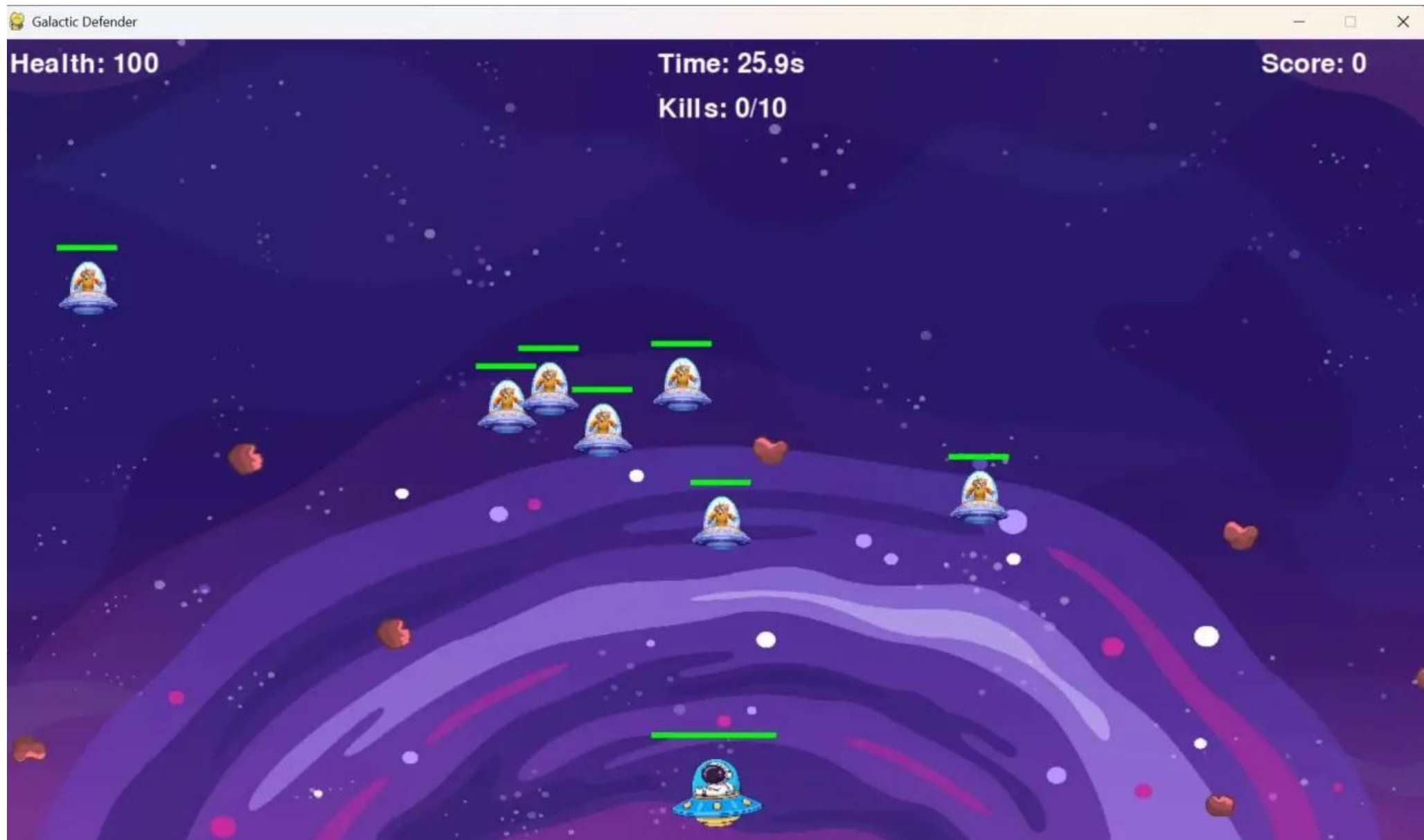
Weights:

- Checkers amount
- Queens amount
- Safe checkers
- Safe queens
- Movable checkers
- Movable queens
- Distance to promotion line

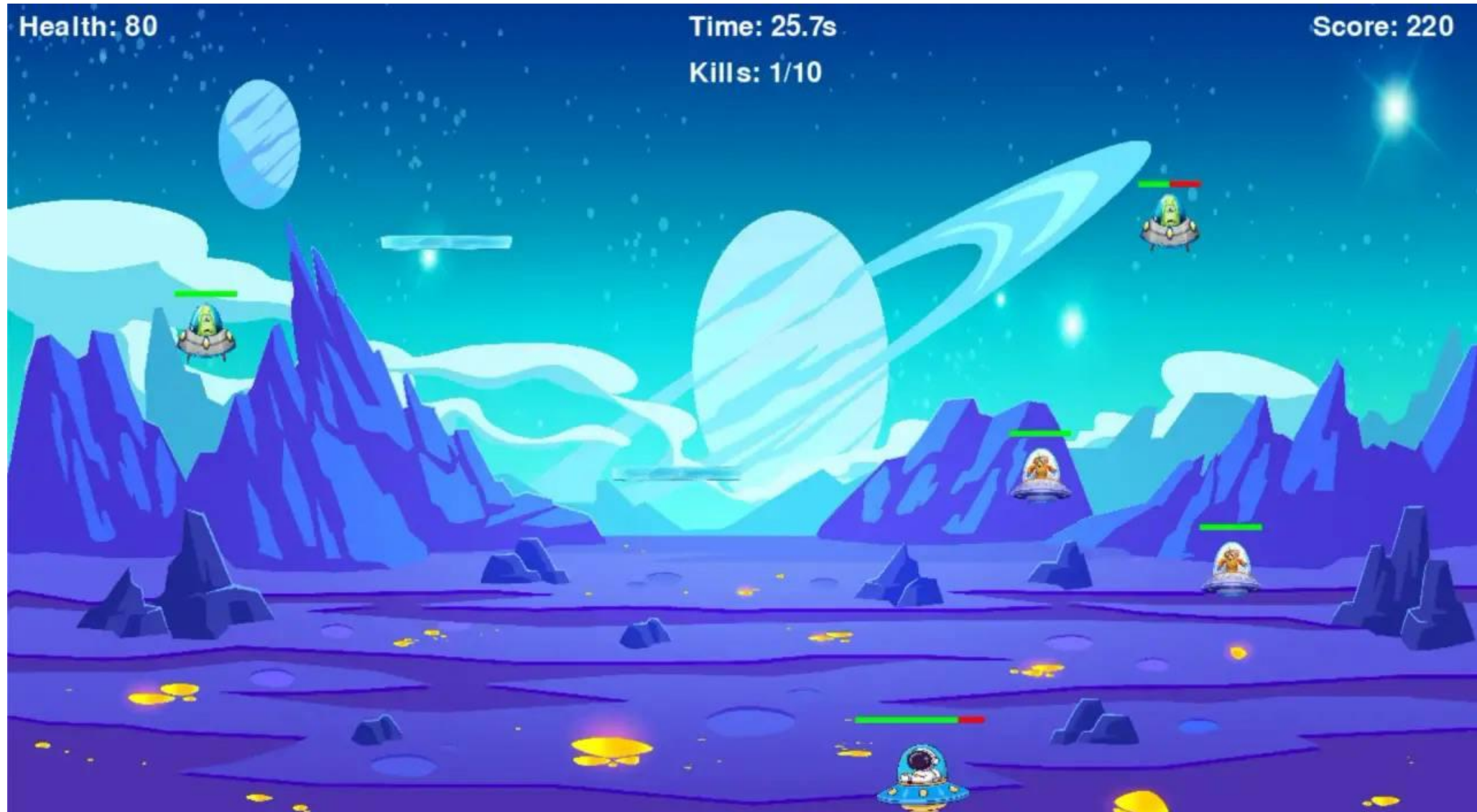
PyGame. Пример 1



PyGame. Пример 2



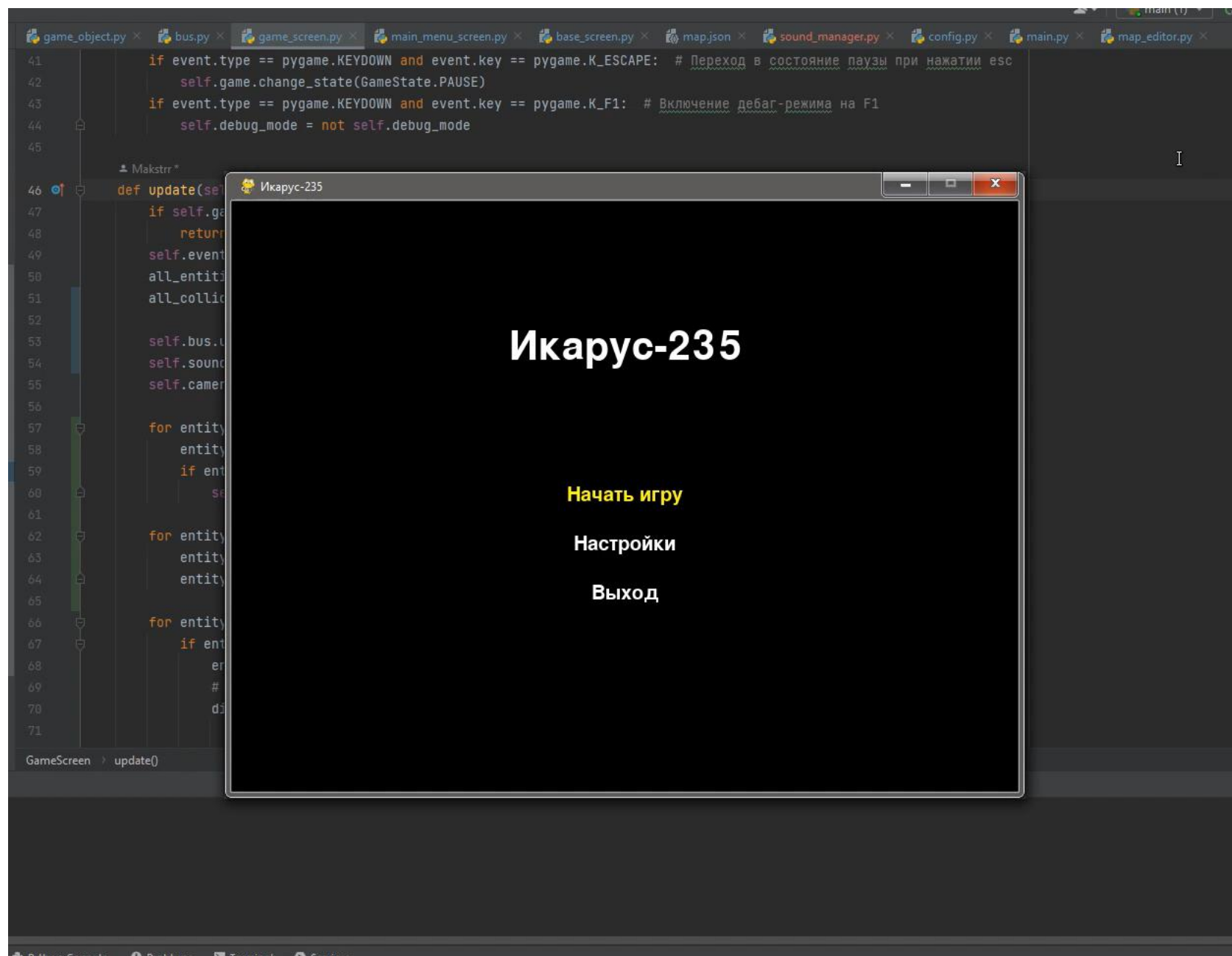
PyGame. Пример 2



PyGame. Пример 2



PyGame. Пример 3



Архитектура

Архитектура – это организация информационной системы.

Эта организация включает все компоненты, то, как они взаимодействуют друг с другом, среду, в которой они работают, а также принципы, используемые для проектирования программного обеспечения.

Архитектура ООП

Использование ООП в PyGame:

- 1. Понятность и читаемость кода.** Код становится более структурированным и логичным.
- 2. Переиспользуемость.** Классы можно переиспользовать в разных частях игры.
- 3. Расширяемость.** Легко добавлять новые типы объектов (например, новые типы врагов) путем создания новых классов, наследуемых от `GameObject`.

Архитектура Семейство Model-View

M-V-...

Model. Содержит бизнес-логику и данные приложения.

Примеры: данные и состояние игрока, положение объектов на карте.

View. Отвечает за отображение данных из модели на экране. Представление получает данные от модели и визуализирует их с помощью PyGame.

Представление уже не должно содержать бизнес-логику.

Примеры: отрисовка игрока, врагов, фона.

M-V-C

Model. Содержит бизнес-логику и данные приложения.

Примеры: данные и состояние игрока, положение объектов на карте.

View. Отвечает за отображение данных из модели на экране. Представление получает данные от модели и визуализирует их с помощью PyGame.

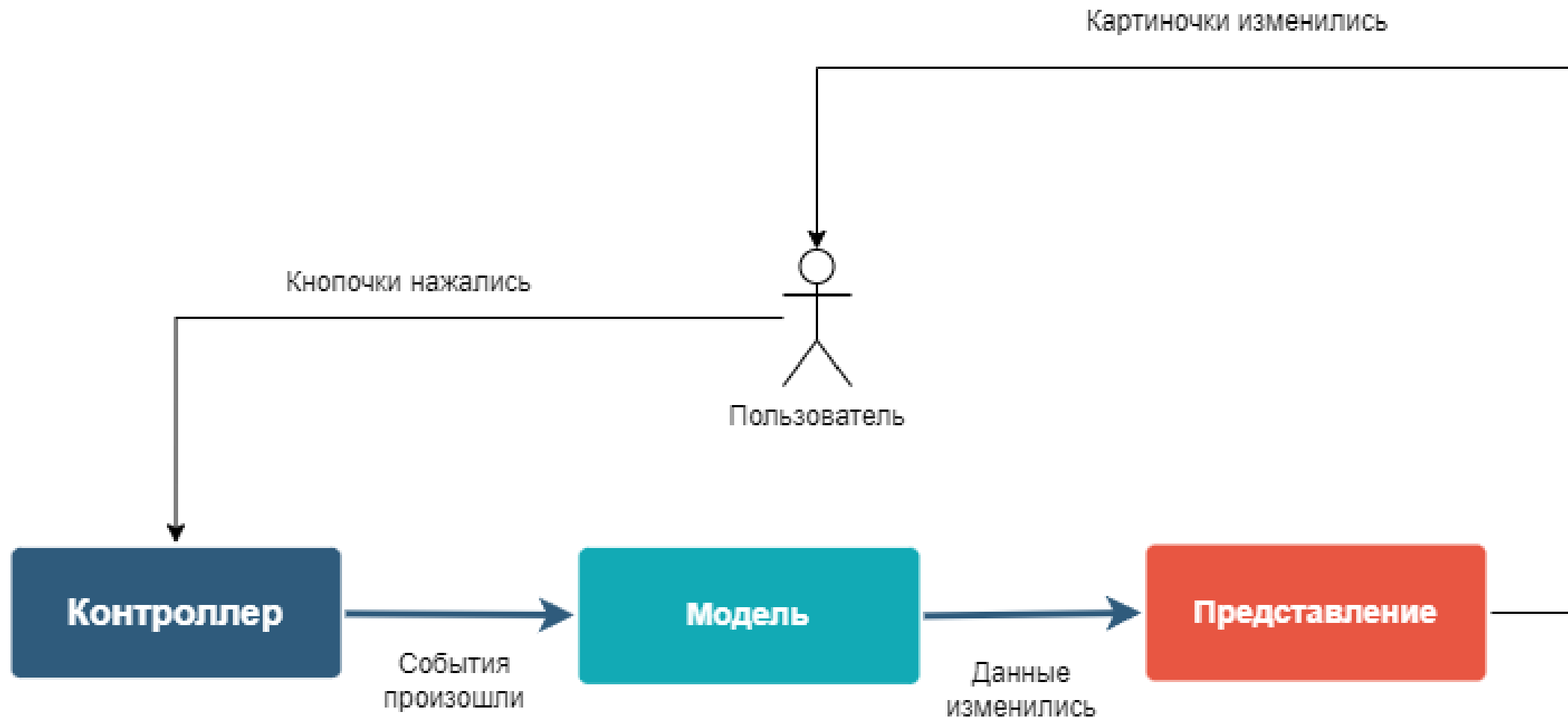
Представление уже не должно содержать бизнес-логику.

Примеры: отрисовка игрока, врагов, фона.

Controller. Сущность, ответственная за обработку действий пользователя и информации в модель. Принимает решения о том, нужно ли (и как) обновлять представление.

Примеры: события нажатия клавиши, ввод пользователя.

M-V-C



<https://gg.xetren.com/ru/mvc>

M-V-P

Model. Содержит бизнес-логику и данные приложения.

Примеры: данные и состояние игрока, положение объектов на карте.

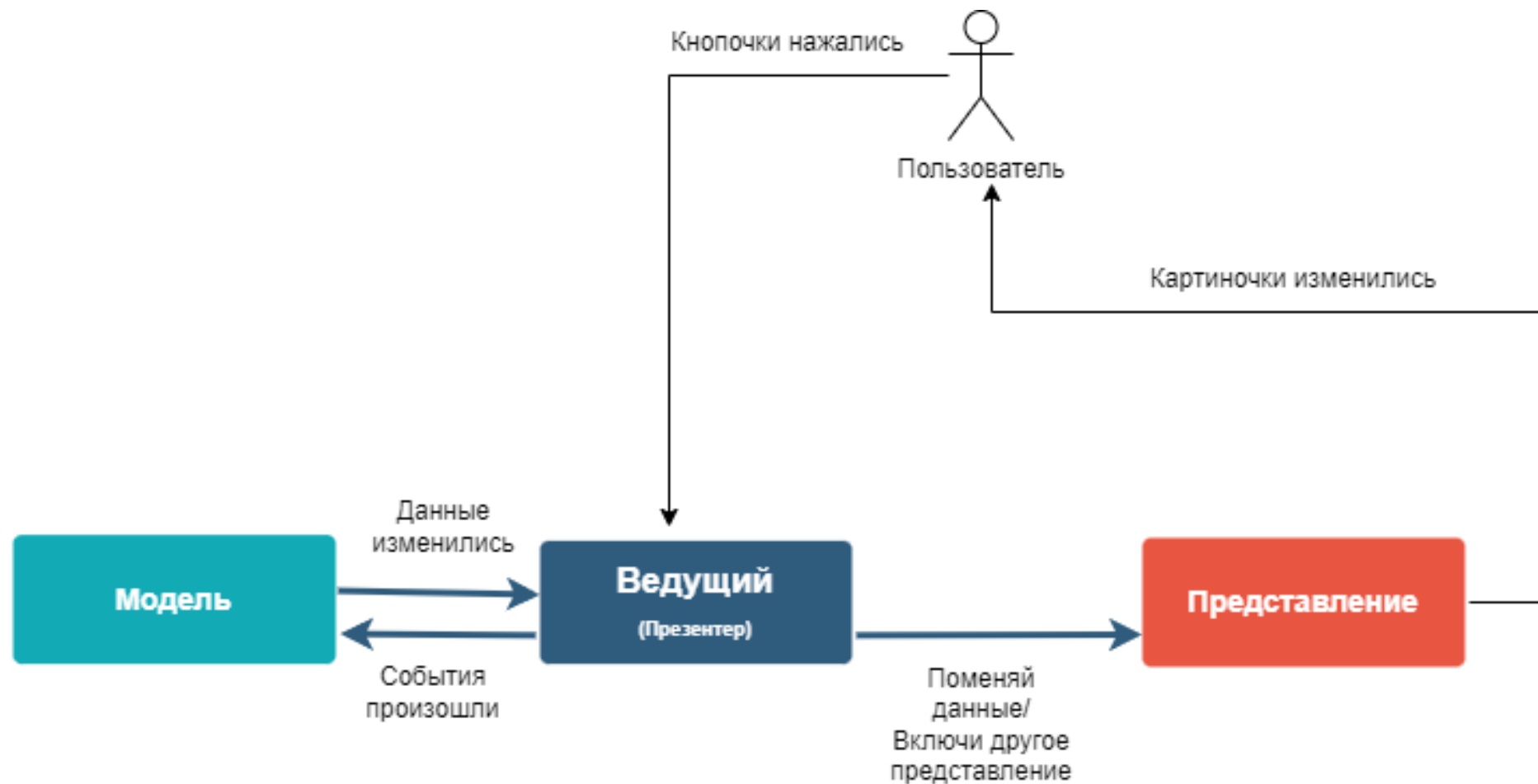
View. Отвечает за отображение данных из модели на экране. Представление получает данные от модели и визуализирует их с помощью PyGame.

Представление уже не должно содержать бизнес-логику.

Примеры: отрисовка игрока, врагов, фона.

Presenter. Является посредником между моделью и представлением. Презентер реагирует на действия пользователя, которые передаются от представления, обрабатывает эти данные (возможно, обновляя модель), и затем обновляет представление. В отличие от контроллера в MVC, презентер содержит значительную часть бизнес-логики, связанной с обновлением представления.

M-V-P



<https://gg.xetren.com/ru/mvc>

M-V-VM

Model. Содержит бизнес-логику и данные приложения.

Примеры: данные и состояние игрока, положение объектов на карте.

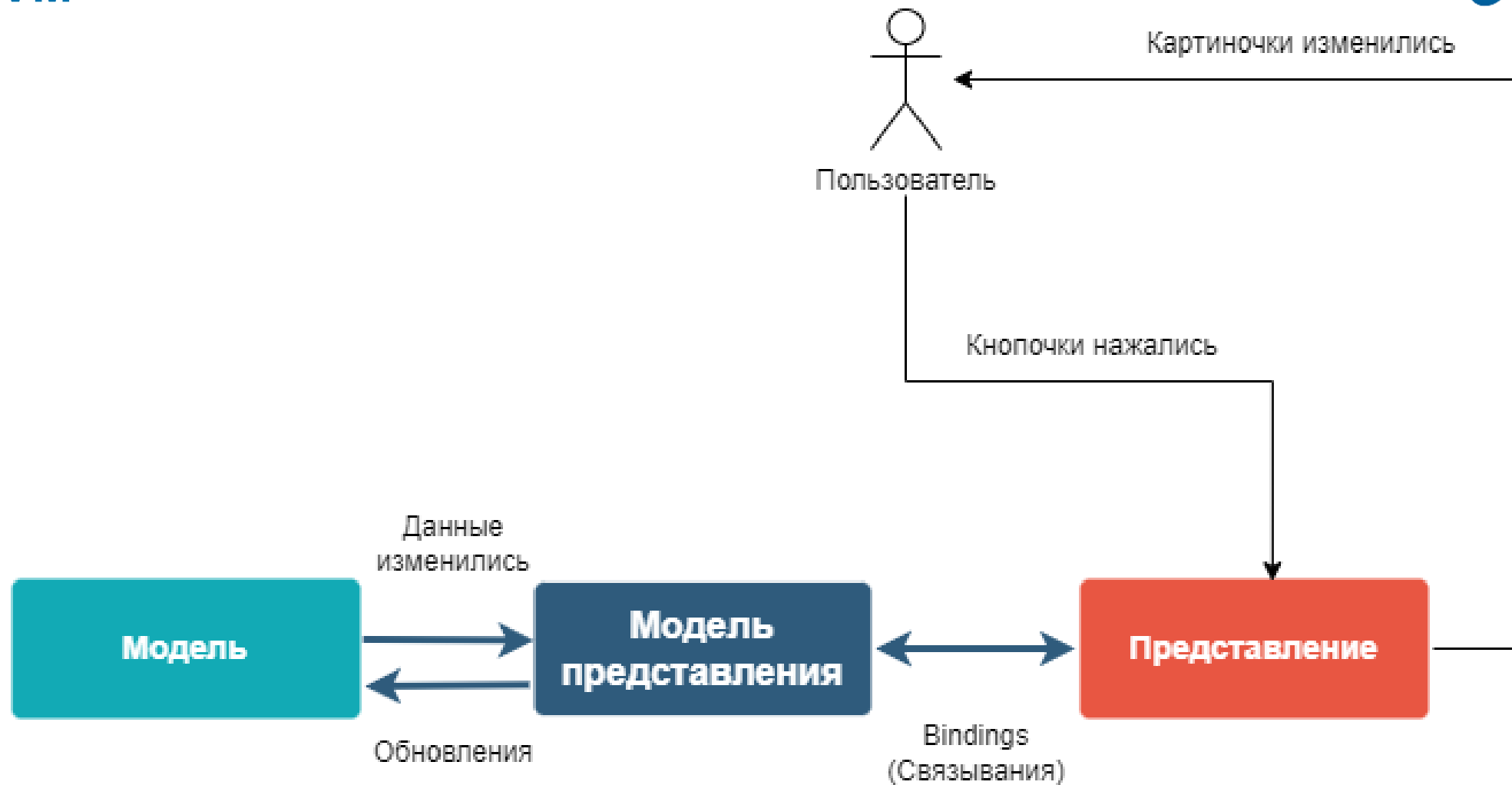
View. Отвечает за отображение данных из модели на экране. Представление получает данные от модели и визуализирует их с помощью PyGame.

Представление уже не должно содержать бизнес-логику.

Примеры: отрисовка игрока, врагов, фона.

ViewModel. Действует как посредник между Model и View. Она содержит логику представления, которая отвечает за обработку команд от View, реагирование на изменения в Model и обновление состояния View через механизмы связывания данных. ViewModel может также включать в себя валидацию данных, выполнение команд и другие аспекты презентационной логики.

M-V-VM



<https://gg.xetren.com/ru/mvc>

Критерии оценивания

F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	
Соответствие кода изученным принципам					Сложность алгоритма (выбирается лишь один столбец, выставляется 0-100)			Корректная, обоснованная реализация архитектуры		Полноценность проекта					Результат	
1	1	3	1	3	3	6	15	10	5	2	2	2	3	2	50	
DRY	KISS	SOLID	Чистота кода: comments duplicate code, Magic Const, etc.	Корректность инкапсуляции, абстракций, наследования	Легкий	Средний	Сложный	Правильно разделена ответственность между компонентами (model не перегружен)	Удобная и простая структура проекта (понятное разбиение на файлы)	Работоспособность (нерабочий проект не оценивается)	Удобство пользования (наличие меню, сохранения прогресса и т.д.)	Тесты	Наличие стабильной гит-истории	Прочее (сюжет, красоты, музыка и т.д.)		

Что еще почитать?



Уральский федеральный университет

имени первого Президента
России Б.Н.Ельцина

**Институт радиоэлектроники
и информационных
технологий — РТФ**

